4.11 KEY TERMS, REVIEW QUESTIONS, AND PROBLEMS

Key Terms

jacketing	multithreading	task
kernel-level thread	port	thread
lightweight process	process	user-level thread
message		

Review Questions

4.1. Table 3.5 lists typical elements found in a process control block for an unthreaded OS. Of these, which should belong to a thread control block and which should belong to a process control block for a multithreaded system?

4.2. List reasons why a mode switch between threads may be cheaper than a mode switch between processes.

4.3. What are the two separate and potentially independent characteristics embodied in the concept of process?

4.4. Give four general examples of the use of threads in a single-user multiprocessing system.

4.5. What resources are typically shared by all of the threads of a process?

4.6. List three advantages of ULTs over KLTs.

4.7. List two disadvantages of ULTs compared to KLTs.

4.8. Define jacketing.

Problems

4.1. It was pointed out that two advantages of using multiple threads within a process are that (1) less work is involved in creating a new thread within an existing process than in creating a new process, and (2) communication among threads within the same process is simplified. Is it also the case that a mode switch between two threads within the same process involves less work than a mode switch between two threads in different processes?

4.2. In the discussion of ULTs versus KLTs, it was pointed out that a disadvantage of ULTs is that when a ULT executes a system call, not only is that thread blocked, but also all of the threads within the process are blocked. Why is that so?

4.3. OS/2 is an obsolete OS for PCs from IBM. In OS/2, what is commonly embodied in the concept of process in other operating systems is split into three separate types of entities: session, processes, and threads. A session is a collection of one or more processes associated with a user interface (keyboard, display, and mouse). The session represents an interactive user application, such as a word processing program or a spreadsheet. This concept allows the personal-computer user to open more than one application, giving each one or more windows on the screen. The OS must keep track of which window, and therefore which session, is active, so that keyboard and mouse input are routed to the appropriate session. At any time, one session is in foreground mode, with other sessions in background mode. All keyboard and mouse input is directed to one of the processes of the foreground session, as dictated by the applications. When a session is in foreground mode, a process performing video output sends it directly to the hardware video buffer and thence to the user's screen.

When the session is moved to the background, the hardware video buffer is saved to a logical video buffer for that session. While a session is in background, if any of the threads of any of the processes of that session executes and produces screen output, that output is directed to the logical video buffer. When the session returns to foreground, the screen is updated to reflect the current contents of the logical video buffer for the new foreground session.

There is a way to reduce the number of process-related concepts in OS/2 from three to two. Eliminate sessions, and associate the user interface (keyboard, mouse, and screen) with processes. Thus, one process at a time is in foreground mode. For further structuring, processes can be broken up into threads.

a. What benefits are lost with this approach?

b. If you go ahead with this modification, where do you assign resources (memory, files, etc.): at the process or thread level?

4.4. Consider an environment in which there is a one-to-one mapping between user-level threads and kernel-level threads that allows one or more threads within a process to issue blocking system calls while other threads continue to run. Explain why this model can make multithreaded programs run faster than their single-threaded counterparts on a uniprocessor computer.

4.5. If a process exits and there are still threads of that process running, will they continue to run?

4.6. The OS/390 mainframe operating system is structured around the concepts of address space and task. Roughly speaking, a single address space corresponds to a single application and corresponds more or less to a process in other operating systems. Within an address space, a number of tasks may be generated and execute concurrently; this corresponds roughly to the concept of multithreading. Two data structures are key to managing this task structure. An address space control block (ASCB) contains information about an address space needed by OS/390 whether or not that address space is executing. Information in the ASCB includes dispatching priority, real and virtual memory allocated to this address space, the number of ready tasks in this address space, and whether each is swapped out. A task control block (TCB) represents a user program in execution. It contains information needed for managing a task within an address space, including processor status information, pointers to programs that are part of this task, and task execution state. ASCBs are global structures maintained in system memory, while TCBs are local structures maintained within their address space. What is the advantage of splitting the control information into global and local portions?

4.7. Many current language specifications, such as for C and C++, are inadequate for multithreaded programs. This can have an impact on compilers and the correctness of code, as this problem illustrates. Consider the following declarations and function definition:

```
int global_positives = 0;
typedef struct list {
    struct list *next;
    double val;
} * list;

void count_positives(list l)
{
    list p;
    for (p = l; p; p = p -> next)
        if (p -> val > 0.0)
            ++global_positives;
}
```

Now consider the case in which thread A performs

```
count_positives(<list containing only negative values>);
```

while thread B performs

```
++global_positives;
```

a. What does the function do?
b. The C language only addresses single-threaded execution. Does the use of two parallel threads create any problems or potential problems?

4.8. But some existing optimizing compilers (including gcc, which tends to be relatively conservative) will "optimize" `count_positives` to something similar to

```
void count_positives(list l)
{
    list p;
    register int r;
r = global_positives;
    for (p = l; p; p = p -> next)
        if (p -> val > 0.0) ++r;
    global_positives = r;
}
```

What problem or potential problem occurs with this compiled version of the program if threads A and B are executed concurrently?

4.9. Consider the following code using the POSIX Pthreads API:

thread2.c

```
#include <pthread.h>
#include <stdlib.h>
#include <unistd.h>
#include <stdio.h>
int myglobal;
    void *thread_function(void *arg) {
        int i,j;
        for ( i=0; i<20; i++ ) {
            j=myglobal;
            j=j+1;
            printf(".");
            fflush(stdout);
            sleep(1);
            myglobal=j;
        }
        return NULL;
}
```

```
int main(void) {
    pthread_t mythread;
    int i;
    if ( pthread_create( &mythread, NULL, thread_function,
         NULL) ) {
        printf(ldquo;error creating thread.”);
        abort();
    }
for ( i=0; i<20; i++) {
    myglobal=myglobal+1;
    printf(“o”);
    fflush(stdout);
    sleep(1);
}
if ( pthread_join ( mythread, NULL ) ) {
    printf(“error joining thread.”);
abort();
}
printf(“\nmyglobal equals %d\n”,myglobal);
exit(0);
}
```

In `main()` we first declare a variable called mythread, which has a type of `pthread_t`. This is essentially an ID for a thread. Next, the `if` statement creates a thread associated with `mythread`. The call `pthread_create()` returns zero on success and a nonzero value on failure. The third argument of `pthread_create()` is the name of a function that the new thread will execute when it starts. When this `thread_function()` returns, the thread terminates. Meanwhile, the main program itself defines a thread, so that there are two threads executing. The `pthread_join` function enables the main thread to wait until the new thread completes.

a. What does this program accomplish?

b. Here is the output from the executed program:

```
$ ./thread2
..o.o.o.o.oo.o.o.o.o.o.o.o.o.o..o.o.o.o.o
myglobal equals 21
```

Is this the output you would expect? If not, what has gone wrong?

4.10. The Solaris documentation states that a ULT may yield to another thread of the same priority. Isn't it possible that there will be a runnable thread of higher priority and that therefore the yield function should result in yielding to a thread of the same or higher priority?

4.11. In Solaris 9 and Solaris 10, there is a one-to-one mapping between ULTs and LWPs. In Solaris 8, a single LWP supports one or more ULTs.

a. What is the possible benefit of allowing a many-to-one mapping of ULTs to LWPs?

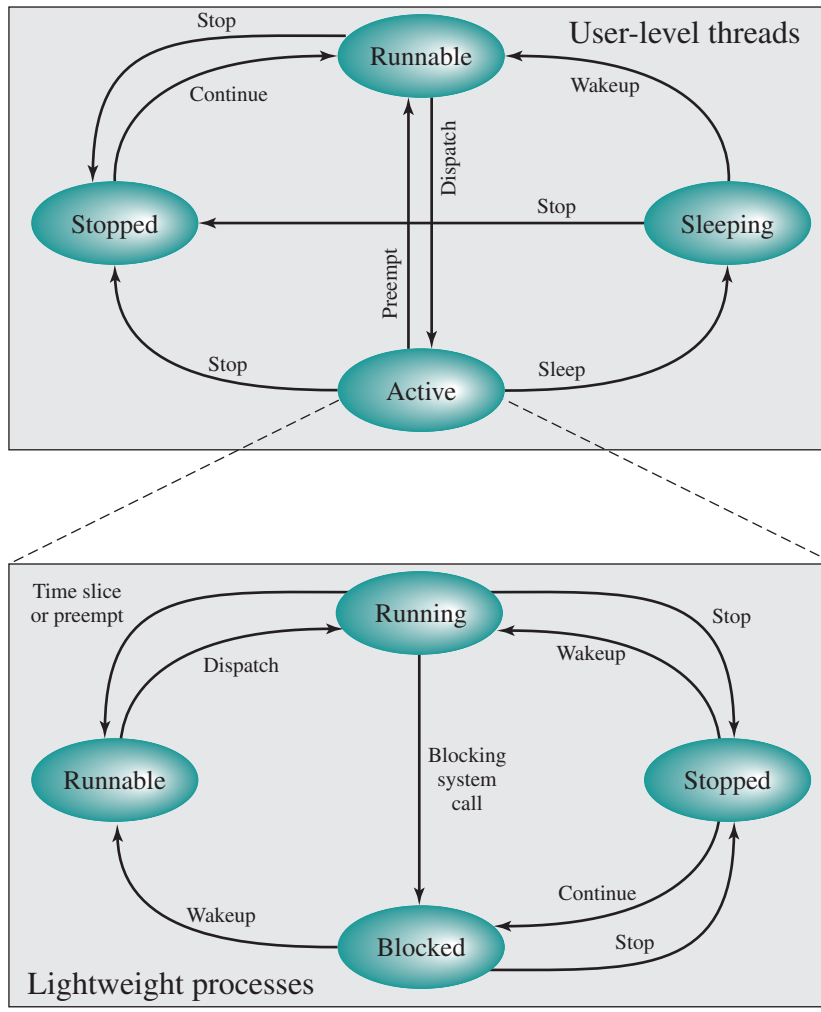

Figure 4.18 **Solaris User-Level Thread and LWP States**

b. In Solaris 8, the thread execution state of a ULT is distinct from that of its LWP. Explain why.

c. Figure 4.18 shows the state transition diagrams for a ULT and its associated LWP in Solaris 8 and 9. Explain the operation of the two diagrams and their relationships.

4.12. Explain the rationale for the Uninterruptible state in Linux.